EECE 7205: Fundamentals of Computer Engineering Multi-Course and Exam Scheduler - Iteration #03

Nashita Hasan, Katherine Tse
Email: hasan.nas@northeastern.edu, tse.kat@northeastern.edu
NUID: 002750189, 003936627

Due: November 22, 2026

1 Project Progress and Scope

1.1 The Problem

- **Problem**: Courses and exams need to be scheduled so that there are not two in the same room and there are not students or professors with overlapping events.
- **Users**: The University Administration Team would use it for general course planning and professors would use it for exam scheduling.

1.2 Features

Feature	Status	Evidence/Remaining Work
Load Input Data	Complete	Reads sample .CSV files and checks fields
Greedy Algorithm	Complete	Handles all test cases; generates appropriate schedule
Build Graph	Complete	Handles all test cases; generates appropriate schedule
Assign Room/Time	Complete	Handles all test cases; generates appropriate schedule
Save Output Data	Complete	Writes resulting .CSV files with final schedule
2nd Core Algorithm	Planned	Selected Backtracking; pending implementation

1.3 Scope Assessment

The scope of the project remains the same, as we are still expected to receive course, professor, and student information and use graph coloring to return a final schedule with no conflicts. In terms of design changes to the algorithm, we have decided to assign rooms and time slots simultaneously in one function rather than having two separate functions establish these assignments. In addition, we have established that there are 12 available time slots for events, which span from 8 am to 8 pm, and 100 rooms available in total.

2 Working C++ Prototype

2.1 Sample Run 1: Normal Test Case

Input:

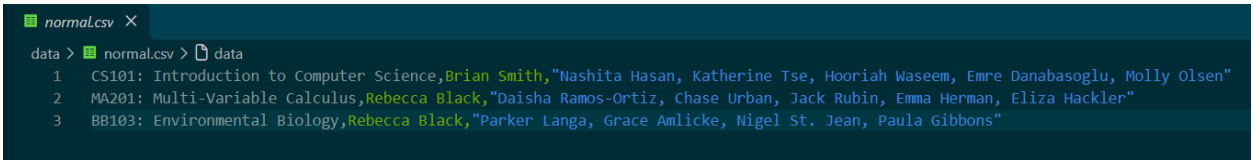


Figure 1: Input for Normal Test Case

Run:

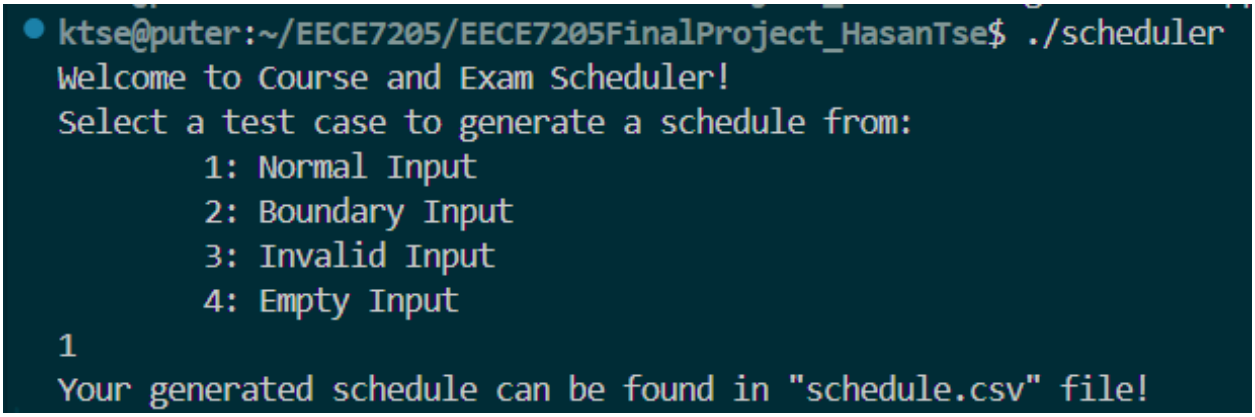


Figure 2: Terminal for Normal Test Case

Output:

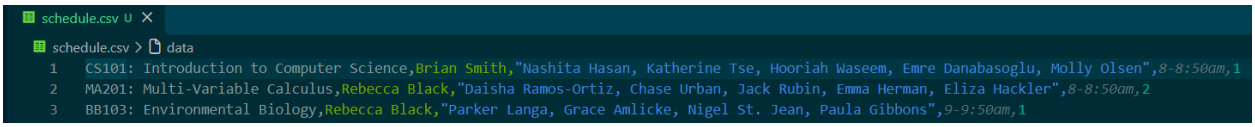


Figure 3: Output for Normal Test Case

2.2 Sample Run 2: Large Test Case

Input: See "large.csv" in the "/data" folder of the GitHub
Run:

```
ktse@puter:~/EECE7205/EECE7205FinalProject_HasanTse$ ./scheduler
Welcome to Course and Exam Scheduler!
Select a test case to generate a schedule from:
        1: Normal Input
        2: Boundary Input
        3: Invalid Input
        4: Empty Input
        5: Large Input
5
Your generated schedule can be found in "schedule.csv" file!
```

Figure 4: Terminal for Large Test Case

Output: See "largeOutput.csv" in the "/data" folder of the GitHub

2.3 Limitations At this time, we recognize the following limitations with our current implementation:

- The program requires that professor and student names be typed exactly the same in order to find a match. This means that if there is an extra space between a first and a last name, or if capitalization is off, the program will not flag the overlap.
- The program does not optimize the time table. This means that the courses will be assigned to the earliest time possible which creates a morning heavy schedule.
- The program currently only runs with a CSV file input. Other file formats are not supported.
- Since the program uses the greedy algorithm there is a chance that the program can report not finding a solution when there is one.

3 Initial Algorithm and Program Design

3.1 Purpose

The purpose of graph coloring using the Greedy Algorithm is to assign each course or exam a time slot and room so that no professors or students have conflicts. Graph coloring assigns a color to each vertex and ensures that no connected vertices have the same color. This corresponds directly with our goal.

3.2 Data Structures

The data structure used is a graph because there are courses (nodes) that can be linked (edges) to a variety of other courses based on a shared professor or student.

3.3 Pseudocode

Algorithm 3.1 Graph Class, and Greedy Coloring

```
1: class Graph
2:     Private:
3:       NumVertices
4:       AdjacencyLists
5:     Public:
6:     function GRAPH(numV)
7:         NumVertices ← numV
8:         AdjacencyLists ← new Vector[numV]
9:     end function
10:    function ADDEDGE(v1, v2)
11:        AdjecancyLists[v1].APPEND(v2)
12:        AdjecancyLists[v2].APPEND(v1)
13:    end function
14:    function GREEDYCOLORING
15:        result ← new Array[NumVertices]
16:        result[0] ← 0
17:        for i ← 1 TO NumVertices - 1 do
18:            result[i] ← -1
19:        end for
20:        availableColors ← new Array[numTimeSlots]
21:        roomsTaken ← new Array[numTimeSlots]
22:        for i ← 1 TO NumVertices - 1 do
23:            availableColors ← {True}
24:            for j ← AdjacencyList[i].Begin() TO AdjacencyList[i].End() do
25:                if result[j] ≠ -1 then
26:                    availableColors[result[j]] = False
27:                end if
28:            end for
```

Algorithm 3.1 Graph Class, and Greedy Coloring [continued]

```
            color
            for color ← 0 to NumVertices - 1 do
                if availableColors[color] = True AND roomsTaken[color] < numRooms
    then
                    Break
                end if
            end for
            result[i] ← color
            roomsTaken[color] ← roomsTaken[color] + 1
        end for
        return result
    end function
end class
```

3.4 Implementation

Our C++ program reads courses from a ".CSV" file and builds a graph where courses that share a professor or student are connected. The *Graph* class and *GreedyColor* function follow the pseudocode directly: each course gets the lowest time slot that doesn't conflict with its neighbors and still has rooms open. The program then assigns each course a time and room and saves the schedule to "schedule.csv."

3.5 Time and Space Complexity

The time and space complexity for the greedy graph coloring algorithm being implemented is as follows:

$$T(n) = O(V^2 + E)$$
$$SpaceComplexity = O(V + E)$$
$$AuxiliarySpace = O(V)$$

Where V represents the number of vertices in the graph, E represents the number of edges, and assuming an adjacency list is used to store the connections.

3.6 Second Algorithmic Approach

In Iteration 4 we will implement a backtracking algorithm for graph coloring and assigning time slots and rooms. This approach solves the problem by assigning events one at a time. It tries each of the 12 time slots and checks whether that slot has no neighbor conflicts and a free room. Once it finds a valid slot, it assigns the room and time and moves on to the next event. If the algorithm reaches an event with no valid slot, it backtracks and reassigns the previous event until there is a schedule that works. The main difference between the greedy algorithm and backtracking is that greedy never changes a decision, while backtracking can.

4 Initial Testing

After coming up with our smaller test inputs, we walked through our programming logic to determine what we expected each time slot and room to be. For the larger input, we talked more generally about what needed to be true for each course for the output to be

correct since the exact numbers would be harder and more tedious to determine.

4.1 Normal Input (PASS)

- Input: 3 total events (courses/exams)

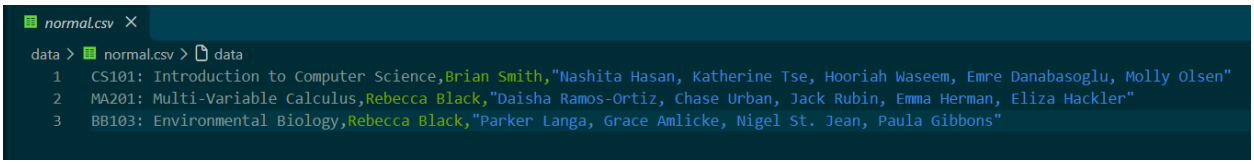


Figure 5: Input for Normal Test Case

- Scenario: Out of the 3 events, 2 of them will share the same professor. The third event has a different professor and no shared students.
- Expected Result: The two events that share the same professor will be assigned different time slots but will be in the same room. The third event has no conflicts, so it will be assigned to the earliest time slot in the second room.
- Actual Result:

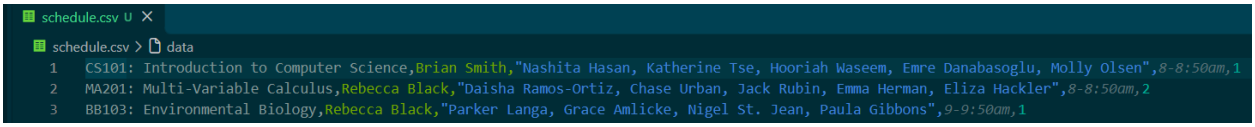


Figure 6: Output for Normal Test Case

4.2 Boundary Case (PASS)

- Input: 3 total events

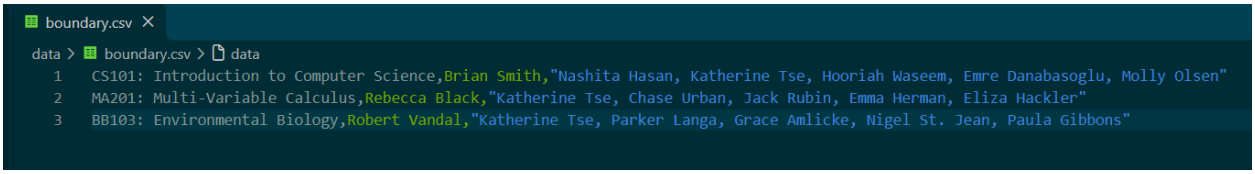


Figure 7: Input for Boundary Test Case

- Scenario: One student is enrolled in all 3 events.
- Expected Result: All events will be assigned different time slots within the same room since no two events share the same time slot.

- Actual Result:

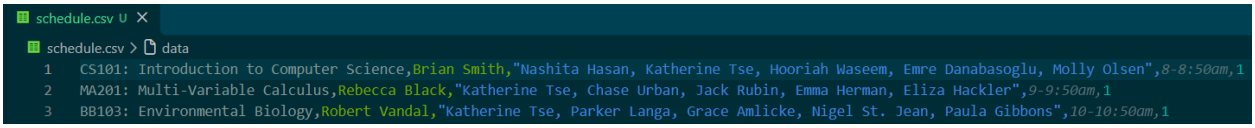


Figure 8: Output for Boundary Test Case

4.3 Invalid Input (PASS)

- Input: 3 events without professors listed

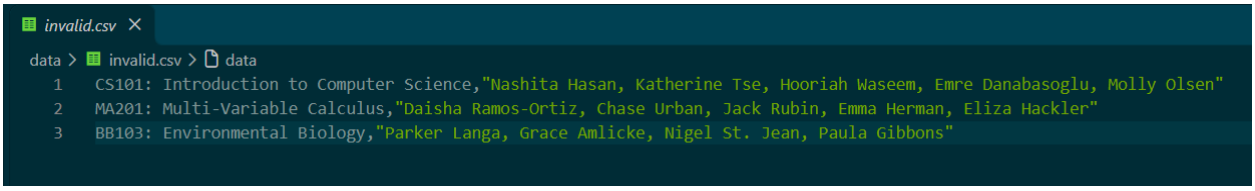


Figure 9: Input for Invalid Test Case

- Scenario: Not enough information given to properly complete task
- Expected Result: An error is returned when reading the file and processing data
- Actual Result:

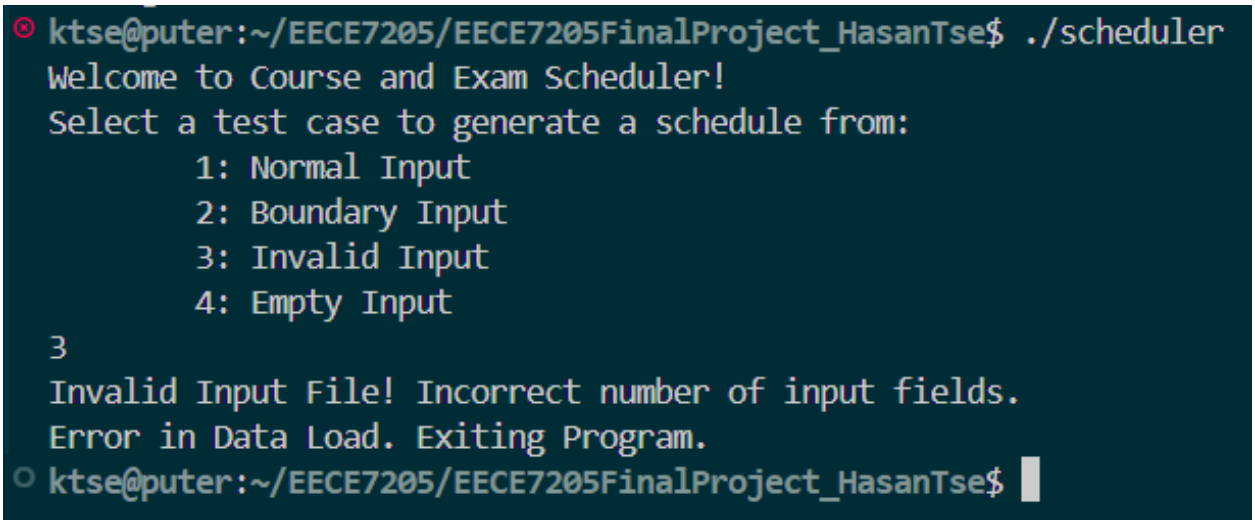


Figure 10: Output for Invalid Test Case

4.4 Empty Input/No-Solution Case (PASS)

- Input: An empty file

- Scenario: No information given/file reads but is empty
- Expected Result: An error is returned when reading the file
- Actual Result:

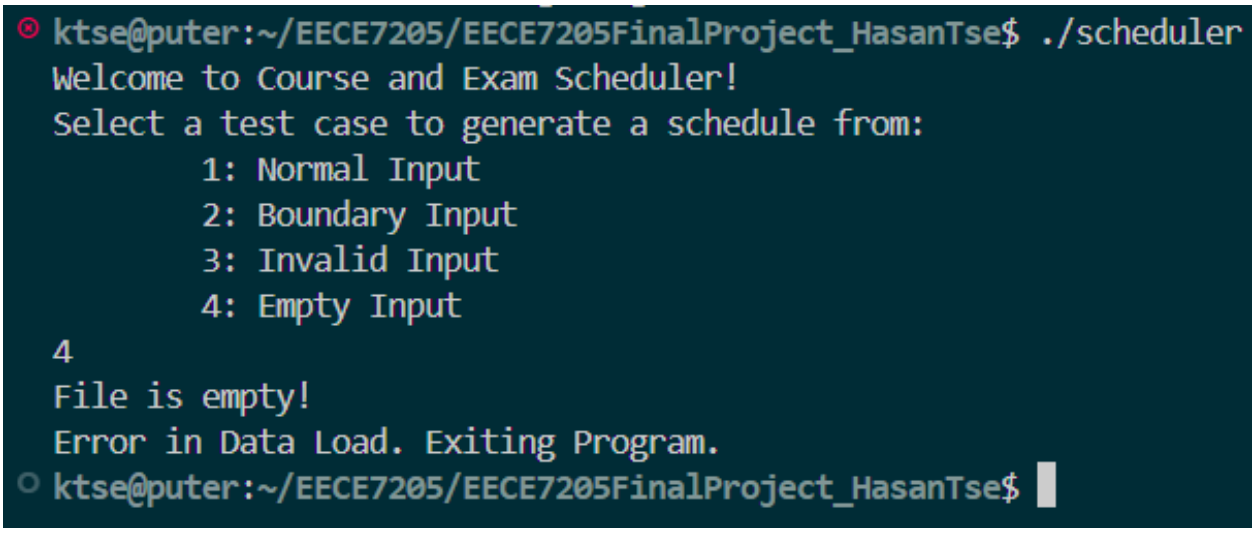


Figure 11: Output for Empty Test Case

4.5 Large Input (PASS)

- Input: 1,000 total events (courses/exams)
  - See "large.csv" in the "/data" folder of the GitHub
- Scenario: Testing large scale data with an average amount of overlapping courses (constrained by total number of courses a professor can teach/a student can take)
- Expected Result: All courses will be assigned a time slot and room and no conflicting courses shall share a time slot.
- Actual Result: See "largeOutput.csv" in the "/data" folder of the GitHub

4.6 Testing Outcomes

When testing the invalid data set, we realized that while we had checks in the load data function, we did not have checks outside of it to exit the rest of the program. This caused the program to crash when we attempted to perform operations on empty data. We were able to add checks on the data so that the program finishes smoothly and communicates what errors (such as invalid inputs or empty files) occurred. Testing allowed us to catch this mistake and add the proper exits into our code. At this time, we have no tests that failed and remain unresolved.

5 GitHub Collaboration

A snapshot of project after iteration 3 can be found under releases on the GitHub page.

GitHub Repository Link:

- https://github.com/Kat24-4/EECE7205FinalProject_HasanTse

Both team members met in person to collaboratively write and discuss code in real time rather than through formal pull requests. Because of this, the commits were made by Katherine Tse as the program was being typed and run on her computer. Since pair coding techniques were used, branches were not utilized since conflicts were not a concern.

6 Individual Contributions and Reflection

Note: Since the group is a team of two, we prefer to do work together to ensure we both have a full and complete understanding of all aspects of the project.

- **Nashita Hasan**:
  - Completed: Pair coded main algorithm (greedy algorithm for graph coloring) and utility functions to build graph, and assign rooms and times to events, worked together to create test inputs, took lead on writing documentation and proofread alongside team member
  - Challenge: One technical challenge that I faced was not being able to fully understand the code for the utility function for building the graph. How I addressed this is I iteratively reviewed the code and broke down the function into smaller pieces to be able to digest it and successfully program a working function.
  - Progress: From Iteration 2, I would say that I have improved my problem-solving and design skills when it comes to understanding a problem and being able to implement a solution that not only makes sense but is optimal for the scenario. I also have improved my technical skills in regards to C++ syntax, logic, and overall understanding of how the program functions.
  - Planned: Meet with team member to complete backtracking algorithm implementation, final program touches and testing, and documentation and presentation materials.
- **Katherine Tse**:
  - Completed: pair coded the main algorithm and (greedy graph coloring) utility functions (build graph, assign times and rooms, load and save data), coded main() and created a testing menu to run all test cases, debugged code throughout testing process, worked together to create test inputs, generated large input using Claude.ai, aided in writing documentation, proofread and added screenshots to documentation, managed GitHub

- Challenge: My biggest technical challenge I faced was figuring out how to alter the basic greedy algorithm to follow real life scheduling constraints. The base greedy coloring algorithm allows for as many colors as there are vertices which is unrealistic. It took much consideration and talking through how we were going to add limits to the colors and how to ensure that an unreasonable amount of rooms were used. We ended up adding a maximum number of colors separate from the number of vertices and an additional check to accommodate a certain number of rooms. These variables allow us to better simulation real world scheduling.
- Progress: So far, this progress has pushed me work more with c++ tools and libraries that I have not had much experience in. One of the main examples of this is using iostream and creating menus to run and test the code. Additionally, file reading has also been a new area of coding that has been interesting to think through. In terms of collaboration, I feel like I am getting a lot more experience in pair coding rather than just working through problems on my own. Additionally, taking lead on GitHub has helped me get more familiar with GitHub features such as releases. Lastly, I have tried to lean into AI debugging a little bit more and have worked on catering my prompts to get the optimal result in cases such as generating an example large data set.
- Planned: Meet with partner to complete backtracking algorithm, final program touches, documentation, and presentation materials

7 Plan for Iteration 4

For Iteration 4, we plan to fully complete the application by adding instructions for the user to be able to upload their own input CSV file. We also plan to re-solve the problem using the backtracking algorithm and then compare the two approaches to determine the most optimal solution. We will conduct further testing on the second algorithm to be able to make these comparisons. Lastly, we will prepare the final documentation and presentation materials.

The remaining programming tasks are planned to be completed by November 19 and the documentation by November 30 by both team members working jointly.

8 AI Assistance and External Resources

We used Claude AI to help debug errors within our code and to generate the input CSV file for our large data test case. We reviewed and understood the debugging suggestions before implementing them ourselves and confirmed each fix by rerunning the failed test case. We verified the AI-generated CSV file by manually following the formatting and confirming that it was correct and readable by our program.

We utilized the following article to help us understand and implement the greedy algorithm for graph coloring.

- https://www.geeksforgeeks.org/dsa/graph-coloring-set-2-greedy-algorithm/